
DICE: Device-level Integrated Circuits Encoder with Graph Contrastive Pretraining

Sungyoung Lee¹ Ziyi Wang² Seunggeun Kim¹ Taekyun Lee¹ Yao Lai³ David Z. Pan¹

Abstract

Pretraining models with unsupervised graph representation learning has led to significant advancements in domains such as social network analysis, molecular design, and electronic design automation (EDA). However, prior work in EDA has mainly focused on pretraining models for digital circuits, overlooking analog and mixed-signal circuits. To bridge this gap, we introduce DICE: Device-level Integrated Circuits Encoder—the first graph neural network (GNN) pretrained via self-supervised learning specifically tailored for graph-level prediction tasks in both analog and digital circuits. DICE adopts a simulation-free pretraining approach based on graph contrastive learning, leveraging two novel graph augmentation techniques. Experimental results demonstrate substantial performance improvements across three downstream tasks, highlighting the effectiveness of DICE for both analog and digital circuits. Our code is available at: <https://github.com/brianlsy98/DICE>.

1 Introduction

Pretraining models through unsupervised learning has demonstrated significant value across various domains, including large language models (LLMs) in language processing [1, 2], vision models in computer vision [3, 4], and graph models in molecular science [5, 6]. These models are applied to various downstream tasks that differ from their original pretraining objectives and consistently exhibit remarkable performance. In particular, pretraining with unsupervised graph representation learning has emerged as a powerful methodology in graph-related fields such as social network analysis [7, 8], healthcare [9–11], molecular science [12–14], and electronic design automation (EDA) [15–18]. Such approaches expand the scope and utility of available graph data through integration with domain knowledge, enabling models to learn robust representations independent of specific tasks.

Integrated circuits (ICs) can be effectively represented as graphs, where circuit components (e.g., gates, transistors, or other devices) and their electrical connections (e.g., wires or nets) are modeled as nodes and edges. This graph-based abstraction enables the application of graph learning methods to EDA in ICs. However, obtaining labeled data in the IC domain is particularly challenging. This is mainly due to the high costs and time required for running complete IC design and validation processes, the limited number of publicly available circuit designs for dataset construction, and the complexity of the design space, which depends on numerous device parameters. To enable pretraining without labels, prior work has explored unsupervised graph representation learning methods, where circuits are represented as graphs with logic gates as nodes and their connections as edges. These approaches have advanced EDA in digital circuit design [16–20].

However, analog circuits cannot be represented using logic gates; instead, they must be described in terms of device components such as transistors, resistors, and capacitors. Moreover, logic gates themselves are composed of these fundamental devices. **Thus, constructing circuit graphs at the**

¹Electrical and Computer Engineering, The University of Texas at Austin

²Computer Science and Engineering, The Chinese University of Hong Kong

³Computer Science, The University of Hong Kong

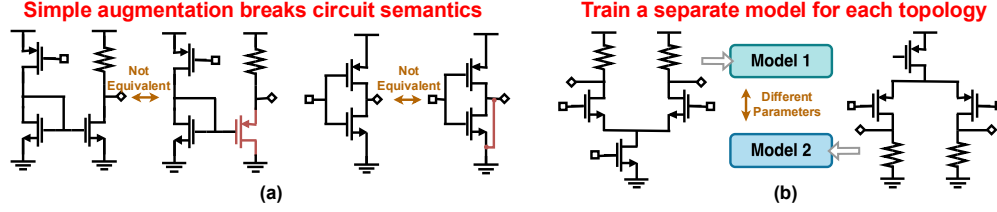


Figure 1: **Key challenges for graph contrastive learning on device-level circuits.** (a) Simple graph augmentations, such as randomly adding edges, can break circuit semantics and result in non-equivalent circuits. (b) Existing approaches require training separate models for each circuit topology, limiting the ability to generalize across diverse structures.

device level offers a more foundational approach to pretraining models applicable to both analog and digital ICs. In this work, we address this fundamental yet underexplored problem.

To effectively apply unsupervised graph representation learning at the device level, a substantial amount of unlabeled circuit data is required. However, even unlabeled data is scarce, as circuits are typically designed using commercial tools that restrict public availability. To address this, graph contrastive learning [21–25] offers a promising solution by generating diverse data through augmentation. Nevertheless, two critical challenges arise when applying it to device-level circuits.

First, traditional graph augmentation techniques—such as randomly adding nodes or edges—are inadequate for device-level circuits. For example, as illustrated in Figure 1(a), simply adding an edge to ground can fix the output voltage at zero, entirely disrupting the original circuit semantics. While certain specialized tools [26, 27] have been effectively used to augment gate-level digital circuits [16, 28], these approaches do not apply to device-level circuit augmentation.

Second, there is a lack of suitable benchmarks for device-level circuits that incorporate diverse circuit structures within each task. As shown in Figure 1(b), prior work typically trains models using a single circuit topology per task [29–33], leading to inefficiencies when multiple topologies correspond to the same task. For instance, when predicting identical performance metrics across different operational amplifier circuits, existing approaches require training separate models for each topology. This limitation in current benchmarks and methodologies hinders the evaluation of pretrained graph models, which should be able to extract robust features across diverse topologies.

Considering these challenges, we propose a graph contrastive learning framework for pretraining **DICE: Device-level Integrated Circuits Encoder**—a graph neural network (GNN) designed for solving graph-level downstream tasks in both analog and digital ICs. Figure 2 provides an overview of our framework. We first convert raw circuit files into graph data using our proposed circuit-to-graph mapping method. Next, we expand the pretraining dataset using two novel data augmentation techniques: positive and negative augmentations, specifically designed for device-level circuits. Then, DICE is pretrained via contrastive learning on graph-level features. After pretraining, DICE is integrated into a full message passing neural network (MPNN) and evaluated on our proposed benchmark. Our benchmark includes three downstream tasks, each involving multiple circuit topologies.

The main contributions of our work are as follows:

- To the best of our knowledge, we propose the first self-supervised graph representation learning framework applicable to both analog and digital circuits. In contrast to prior work that focuses solely on digital circuits, our approach provides a more fundamental method for pretraining graph models that generalize to both analog and digital ICs.
- We introduce two novel data augmentation techniques: positive augmentation, which preserves, and negative augmentation, which perturbs the graph-level functionality of circuits. Both techniques are simulation-free and do not rely on any EDA tools, yet they ensure that the augmented circuits remain valid for simulation by preserving circuit connectivity.
- We present a benchmark comprising three graph-level prediction tasks and demonstrate that DICE is effective for both analog and digital circuits. The first task spans both analog and digital domains, the second focuses on digital circuit operations, and the third targets analog circuit operations. None of the tasks is restricted to a single circuit topology, allowing for evaluation across diverse topologies.

2 Preliminaries

2.1 Graph Contrastive Learning

Graph contrastive learning is one of the major class of unsupervised graph representation learning methods. These approaches learn node, edge, or graph embeddings by maximizing the similarity between positive (similar) samples and minimizing it between negative (dissimilar) samples. To learn robust representations applicable to various tasks, they rely on data augmentation to diversify training samples. Augmentations for graph include adding nodes or perturbing edges within the graph.

One of the most widely used objectives in contrastive learning is the NT-Xent loss [34]. Given a dataset of N samples, each sample is augmented twice to produce $2N$ samples, forming N positive pairs. For each sample $i \in 1, 2, \dots, 2N$, let $\mathbf{z}_i$ denote its embedding and $\mathbf{z}_{j(i)}$ denote the embedding of its positive counterpart. The NT-Xent loss is then defined as:

$$\mathcal{L}_{\text{NT-Xent}} = \frac{1}{2N} \sum_{i=1}^{2N} -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_{j(i)})/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \quad (1)$$

$\text{sim}(\cdot, \cdot)$ denotes a similarity function (e.g., cosine similarity), and τ is a temperature coefficient, typically set to 0.1. Minimizing Eq. (1) encourages the embeddings of positive pairs to cluster in the feature space while pushing apart the embeddings of negative pairs.

There are also approaches that maximize the similarity between positive pairs without relying on negative samples. SimSiam [35] is one such method, and the objective is given by:

$$\mathcal{L}_{\text{SimSiam}} = -\frac{1}{2} \left(\frac{\mathbf{p}_1^\top \text{sg}(\mathbf{z}_2)}{\|\mathbf{p}_1\|_2 \cdot \|\text{sg}(\mathbf{z}_2)\|_2} + \frac{\mathbf{p}_2^\top \text{sg}(\mathbf{z}_1)}{\|\mathbf{p}_2\|_2 \cdot \|\text{sg}(\mathbf{z}_1)\|_2} \right) \quad (2)$$

Here, $\mathbf{p}_1$ and $\mathbf{p}_2$ are the outputs of the predictor MLP given the feature inputs $\mathbf{z}_1$ and $\mathbf{z}_2$, respectively. The operator $\text{sg}(\cdot)$ denotes the stop-gradient operation, and $\|\cdot\|_2$ represents the L2 norm. The pair $(\mathbf{z}_1, \mathbf{z}_2)$ always constitutes a positive pair, and Eq. (2) maximizes their cosine similarity accordingly.

In graph-based settings, contrastive learning can be applied at three levels: node, edge, and graph. Depending on the target task, the inputs $\mathbf{z}$ in Eq. (1)-(2) can represent node, edge, or graph embeddings. In this work, we pretrained DICE using graph embeddings.

2.2 Graph Representation Learning for ICs

Digital circuits can be fully constructed using logic gates, and they form directed acyclic graphs (DAGs) [36]. With these graphs, pioneering works have pretrained models via self-supervised learning to reduce the excessive time and cost associated with large-scale digital hardware design. Methods in [18, 19] pretrain models by comparing logic functionalities and then transferring the models to downstream tasks such as logic synthesis and boolean satisfiability (SAT) solving. Moreover, graph contrastive learning is employed in [16, 17] to pretrain models that cluster functionally equivalent circuits, with applications to downstream tasks such as arithmetic block identification and netlist classification. To address data scarcity, these works utilize logic synthesis tools such as Yosys [26] and ABC [27], which generate augmented circuit views that preserve identical functionalities. Recently, LLMs have also been explored as an approach for augmenting digital circuit data [37, 38], and these models have been further utilized for multimodal circuit representation learning [39].

Analog and mixed-signal (AMS) circuits must be represented using device components, as they cannot be constructed solely with logic gates. Using device-level circuit graphs, the supervised learning approach in [40] pretrains a GNN to predict DC voltage outputs, addressing a node-level prediction task for analog circuits. The learned representations are subsequently transferred to predict other simulation metrics, such as gain and bandwidth of operational amplifier circuits. However, self-supervised pretraining methods remain underexplored, in part due to the lack of reliable data augmentation techniques for AMS circuits. Although prior work has proposed augmentation strategies for AMS circuits [41, 42], these augmentations are task-specific and primarily focused on analog circuits. LLMs offer an alternative, as they are capable of generating AMS circuits at the device level [43–45], but their effectiveness as an augmentation tool remains uncertain.

3 Method

Building a graph contrastive learning framework for device-level circuits presents several challenges. First, it requires a suitable graph construction method that transforms device-level circuits into graph-structured data. Second, an effective graph augmentation technique that incorporates IC-specific domain knowledge is essential. Third, a comprehensive contrastive learning framework for pretraining, along with diverse downstream tasks for evaluation, must be established. We review work related to these challenges in Appendix A.

Addressing these challenges, we propose a graph contrastive learning framework for pretraining **DICE: Device-level Integrated Circuits Encoder**. In the following subsections, we first introduce our graph construction method for device-level circuits (Section 3.1), then describe our customized data augmentation approach that leverages circuit-specific domain knowledge (Section 3.2). Next, we detail our contrastive learning process using the proposed augmentation scheme (Section 3.3), and explain how we integrate the pretrained model DICE to solve three downstream tasks (Section 3.4).

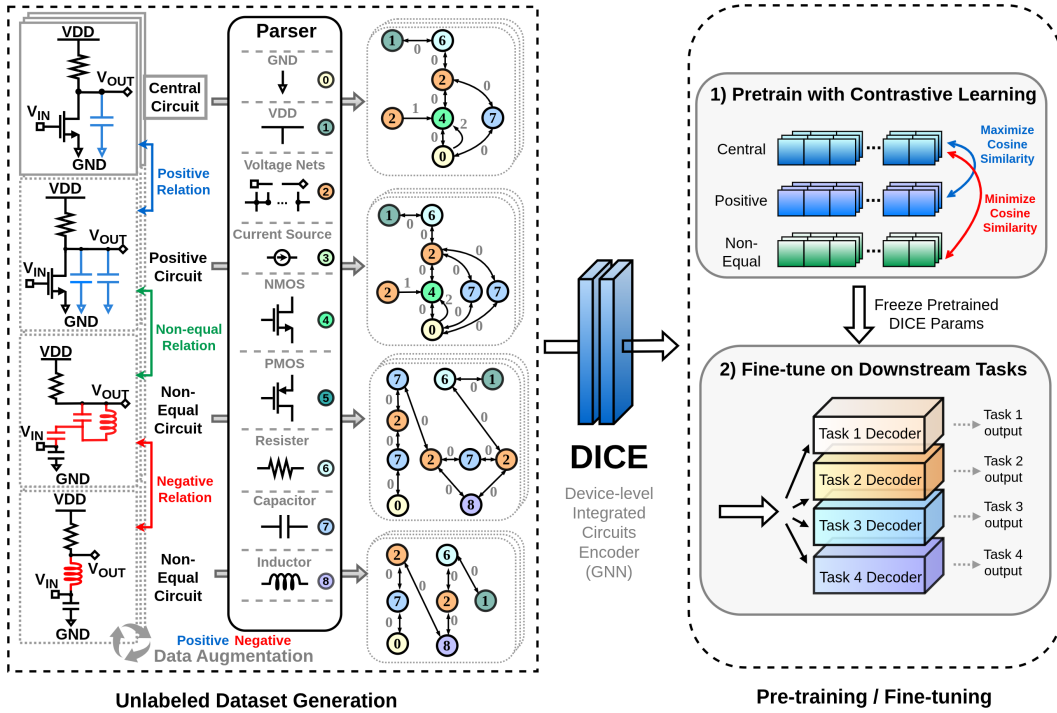


Figure 2: **Overview of our graph contrastive learning framework for pretraining DICE.** We maximize the cosine similarity between circuits that are in positive relation and minimize it between circuits with a non-equal relation. A detailed explanation of these relations is provided in Section 3.3.

3.1 Graph Construction

Considering prior work stated in Appendix A.3, we propose a new method converting device-level circuit into graph. The left part of Figure 2 illustrates our method. We begin by using one-hot encoding to represent various device types and their connections, forming a homogeneous graph. There are nine distinct node types: ground voltage net (0), power voltage net (1), other voltage nets (2), current source (3), NMOS transistor (4), PMOS transistor (5), resistor (6), capacitor (7), and inductor (8). The relationships among these devices are categorized into five edge types: current flow path (0); connections from voltage nets to NMOS gates (1) or bulks (2); and connections from voltage nets to PMOS gates (3) or bulks (4). Notably, the edges from gate and bulk voltage nets to MOSFETs are directed, as these nets exert significantly greater influence on the device than vice versa. With this design, our graph construction method fully leverages edge features without separating device pin nodes from their corresponding device nodes.

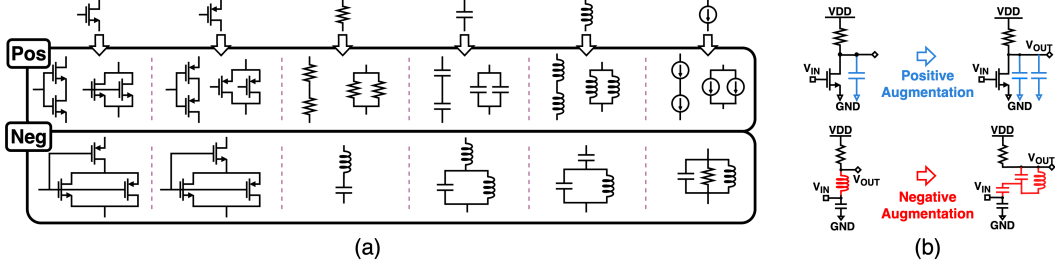


Figure 3: **Rules (a) and examples (b) for positive and negative data augmentation.** Positive augmentation adds an identical device in parallel or series, preserving the overall circuit function. Negative augmentation replaces a subgraph, resulting in a circuit with different overall functionality.

3.2 Positive and Negative Data Augmentation

To address the limitations of including device parameters in the pretraining dataset (see Appendix A.4), we introduce two data augmentation methods: positive and negative augmentation. These methods are used to generate the pretraining dataset without incorporating device parameters. In both methods, subgraphs corresponding to devices within each circuit are modified, as illustrated in Figure 3(a).

Positive augmentation generates new circuit topologies while preserving the graph-level functionality of the original circuit. In each iteration, a random device is selected, and an identical device is added either in parallel or in series. Although device parameters are not explicitly considered, this modification is functionally equivalent to altering the parameters of the selected device (see Appendix B.1). As a result, the augmented graphs retain graph-level properties that closely resemble those of the original. Samples generated through positive augmentation can be further augmented to produce additional positive or negative samples. By maximizing the similarity between these positive samples, DICE learns to capture the underlying semantics shared across different circuit topologies.

However, using only positive augmentation severely limits the diversity of graph-level functionalities in the pretraining dataset. For example, if there are 50 distinct original circuits prior to augmentation, positive augmentation will generate new topologies whose graph-level properties remain constrained to these 50 functional types. As a result, the pretraining dataset would contain graph data representing only 50 unique graph-level functionalities. To address this, we introduce a negative data augmentation technique designed to increase the functional diversity within the pretraining dataset. Specifically, we randomly select a device subgraph and modify it into an alternative subgraph, as illustrated in Figure 3(a). This transformation induces significant changes in frequency response or DC voltage behavior, resulting in new topologies with distinct graph-level functionalities (see Appendix B.2).

3.3 Graph Contrastive Pretraining

Once the augmented pretraining dataset is prepared, graph contrastive learning is performed using graph-level features of the circuits, as illustrated in the upper right part of Figure 2.

Positive, Negative, and Non-Equal Relations. We define three types of relationships between circuits in the pretraining dataset: positive, negative, and non-equal. First, two circuits are in a positive relation if both are generated via positive augmentation from the same original circuit. Second, two circuits are in a negative relation if one is generated through negative augmentation of the original circuit from which the other is derived. Third, all remaining pairs fall under the non-equal relation, representing circuits that are augmented from different original circuits.

For example, consider two different original circuits, x_1 and x_2 , prior to augmentation. Let x_{1+} , x_{2+} denote circuits generated via positive augmentation from x_1 and x_2 , respectively. Also, let x_{1-} and x_{2-} denote circuits generated via negative augmentation. In this case, positively related pairs include (x_1, x_{1+}) and (x_2, x_{2+}) . Negatively related pairs include (x_1, x_{1-}) , (x_2, x_{2-}) , (x_{1+}, x_{1-}) , and (x_{2+}, x_{2-}) . Non-equal pairs include (x_1, x_2) , (x_1, x_{2+}) , (x_1, x_{2-}) , (x_2, x_{1+}) , and (x_2, x_{1-}) .

Our graph contrastive learning framework maximizes the cosine similarity between positive pairs and minimizes it only between non-equal pairs. We do not minimize the similarity between negative pairs, as these circuits differ only in localized structures and still share similar overall topologies compared to non-equal pairs. Empirical results indicate that minimizing the similarity between negative pairs

leads to unstable pretraining. However, negative augmentation still contributes to samples included in non-equal pairs and provide diversity in graph-level functionalities.

Model Architecture of DICE. We define DICE as a pretrained Graph Isomorphism Network with a depth of 2, incorporating edge feature updates (see Appendix C.1). However, any other GNN architecture can be used. All feature update rules used in this work are detailed in Appendix C.

Contrastive Learning Objective. We introduce a loss variant of Eq.(1), formulated in Eq.(3).

$$\mathcal{L}_{DICE}(\theta) = -\mathbb{E}_{x \sim p_{data}} \left[\frac{1}{N^+} \sum_{x' \in X^+(x)} \log \frac{\exp(f_\theta(x, x')/\tau)}{\sum_{x^+ \in X^+(x)} \exp(f_\theta(x, x^+)/\tau_p) + \sum_{x^\neq \in X^\neq(x)} \exp(f_\theta(x, x^\neq)/\tau_n)} \right] \quad (3)$$

The GNN feature extractor f_θ , parameterized by θ , computes the cosine similarity between the graph features of two input graphs. The temperature coefficients are denoted by τ , τ_p , and τ_n , and p_{data} represents the probability distribution over all circuits in the pretraining dataset. Let $X^+(x)$ denote the discrete set of samples that are in a positive relation with x , and $X^\neq(x)$ denote the set of samples that are in a non-equal relation with x . We use N^+ to represent the number of samples in the set $X^+(x)$. Details on the practical implementation of this objective are provided in Appendix D.1.2.

3.4 Application to Downstream Tasks

After pretraining, we integrate DICE into the full message passing neural network (MPNN) model illustrated in Figure 4 to address diverse downstream tasks.

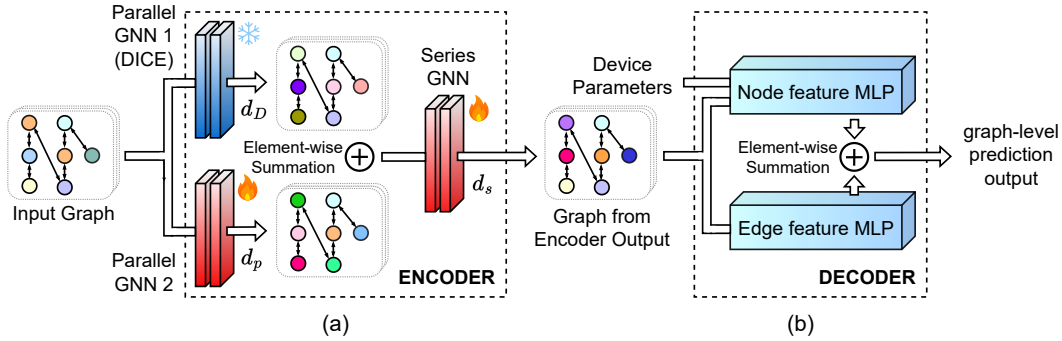


Figure 4: **Encoder (a)–Decoder (b) model architecture used for solving downstream tasks.** For Parallel GNN 2 and Series GNN, both node and edge features are updated according to Eqs. (4)–(6).

The encoder (Figure 4(a)) comprises three GNN modules: two parallel networks that process the same graph input, followed by a series-connected network. One of the parallel GNNs is initialized and frozen with the pretrained DICE parameters, while the other parallel GNN and the series-connected GNN remain fully trainable. The encoder configuration remains fixed for each downstream task if the GNN depths are specified in advance. We denote the GNN depths of the two parallel networks and the series-connected network as d_D , d_p , and d_s , respectively.

The decoder network (Figure 4(b)) takes the output graph from the encoder as input and incorporates device parameters. To encode these parameters, each device type is represented by a 9-dimensional one-hot vector and scaled by its corresponding parameter values. Because parameter values can vary significantly in magnitude across device types, we apply a negative logarithmic transformation prior to scaling. The resulting encoded parameters are then concatenated with the node features from the encoder output and passed through the subsequent layers of the decoder.

4 Experiments

We begin by outlining the experimental setup and presenting the results of pretraining. We then evaluate the pretrained model on three downstream tasks, each formulated as a graph-level prediction problem. Experimental details are provided in Appendix D.

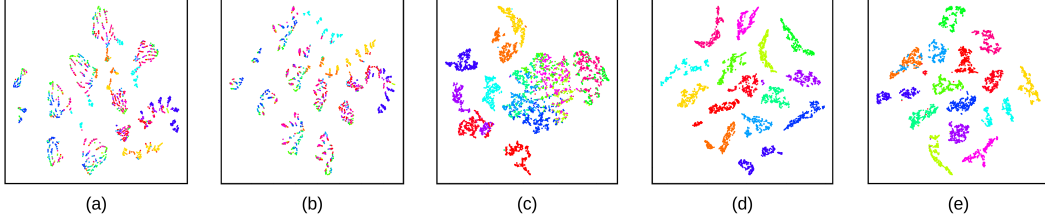


Figure 5: **t-SNE visualization of graph-level features.** Each plot represents: (a) initial embeddings, embeddings processed by an (b) untrained GNN, (c) GNN pretrained with $\mathcal{L}_{\text{SimSiam}}$, (d) GNN pretrained with $\mathcal{L}_{\text{NT-Xent}}$, and (e) GNN pretrained with $\mathcal{L}_{\text{DICE}}$ (DICE).

4.1 Pretraining

Setup. We compare three different contrastive learning approaches: The first approach uses $\mathcal{L}_{\text{SimSiam}}$ (Eq. (2)) with a pretraining dataset generated solely through positive data augmentation. The second approach uses $\mathcal{L}_{\text{NT-Xent}}$ (Eq. (1)), also with a training dataset constructed using only positive augmentation. The third approach is our proposed method described in Section 3, which uses $\mathcal{L}_{\text{DICE}}$ (Eq. (3)) and incorporates both positive and negative data augmentation. For $\mathcal{L}_{\text{NT-Xent}}$, we set $\tau = 0.05$. For $\mathcal{L}_{\text{DICE}}$, we set $(\tau, \tau_p, \tau_n) = (0.05, 0.2, 0.05)$. All GNNs follow Eqs. (4)–(6).

t-SNE Visualization. Figure 5 presents the t-SNE plots of graph embedding vectors. We use the pretraining test dataset (Appendix D.1) constructed solely with positive data augmentation, as negatively augmented samples cannot be clearly categorized and may distort the visualization. Each of the 15 colors represents a distinct initial circuit, and all of its positively augmented views share the same color. Figure 5(c), (d), and (e) exhibit improved clustering compared to Figure 5(a) and (b), providing evidence of effective contrastive pretraining.

Cosine Similarity Values. To further validate the pretraining results, we computed cosine similarity values between the extracted graph-level features of circuits in the test dataset. For this analysis, we used the pretraining test dataset (Appendix D.1) constructed with both positive and negative data augmentation. Table 1 presents the average cosine similarities among pairs in positive, non-equal, and negative relations. The results using DICE confirm that similarities are high between positively related pairs and effectively minimized between non-equal pairs. Also, cosine similarity is also minimized between samples in a negative relation, even without direct minimization in Eq. (3).

Table 1: **Cosine similarity values (mean and standard deviation) between graph embeddings in the pretraining test dataset.** With DICE, similarities are minimized between negative pairs.

Pairs	Initial	Untrained GNN	Pretrained GNN with $\mathcal{L}_{\text{SimSiam}}$	Pretrained GNN with $\mathcal{L}_{\text{NT-Xent}}$	DICE
Positive	0.998 ± 0.002	0.995 ± 0.006	0.804 ± 0.393	0.958 ± 0.080	0.906 ± 0.161
Non-equal	0.993 ± 0.005	0.982 ± 0.016	0.124 ± 0.708	0.481 ± 0.350	0.349 ± 0.491
Negative	0.997 ± 0.002	0.993 ± 0.006	0.505 ± 0.636	0.861 ± 0.175	0.177 ± 0.362

4.2 Downstream Tasks

To support multiple topologies per task (see Appendix A.1 and Appendix A.2), we propose a benchmark consisting of three new graph-level prediction tasks: (1) predicting the relative similarity among fifty different circuits, (2) performance prediction for five delay line circuits, and (3) performance prediction for five operational amplifier circuits. Detailed descriptions are provided in Appendix D.2.

Baseline Setup. Our baselines include prior work that (1) explicitly defines graph construction using transistors, (2) employs GNNs, and (3) predicts circuit simulation results [46, 47]. Specifically, we selected the unpretrained GNN model (**ParaGraph**) from [46], and both the unpretrained (**DeepGen_u**) and pretrained (**DeepGen_p**) GNN models from [47] as baselines. DeepGen_p is pretrained on a supervised DC voltage prediction task, as proposed in [47] (see Appendix D.3). To solve the three downstream tasks, the GNN models from the selected baselines are used as encoders (Figure 4(a)), while the decoder component (Figure 4(b)) remains identical across all models, including ours.

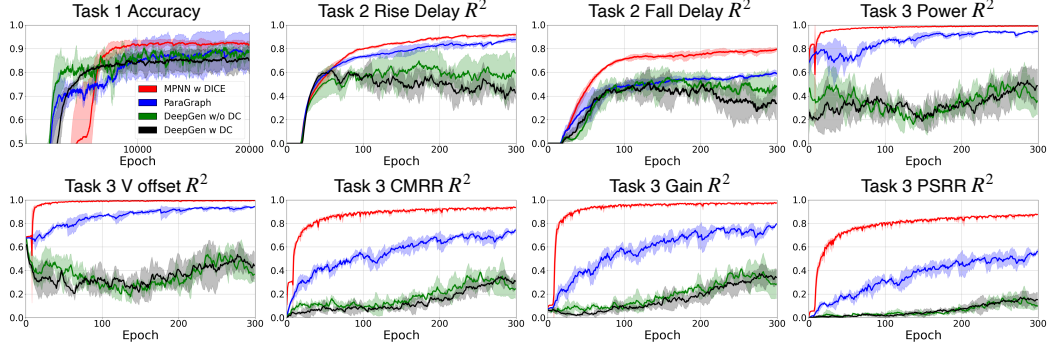


Figure 6: **Performance curves during downstream task training.** Each plot shows the $\mu \pm \sigma$ range across 3 different seeds. We compared our method (red), ParaGraph (blue), DeepGen_u (green), and DeepGen_p (black). Applying DICE leads to stable training, and consistently outperforms all others.

Table 2: **Baseline performance on the downstream tasks.** For each evaluation metric, the best results are highlighted in bold. The encoder incorporating DICE outperforms all baseline models.

Baseline encoders	Task 1 (%)	Task 2 (R^2)		Task 3 (R^2)				
	Accuracy	Rise Delay	Fall Delay	Power	V _{offset}	CMRR	Gain	PSRR
ParaGraph	91.42	0.9255	0.6373	0.9312	0.9585	0.7912	0.8488	0.6215
DeepGen _u	85.02	0.8866	0.6715	0.8403	0.8478	0.5637	0.6813	0.3820
DeepGen _p	84.65	0.9131	0.6875	0.4479	0.5190	0.3246	0.3454	0.0851
DICE	93.94	0.9468	0.8085	0.9916	0.9950	0.9374	0.9769	0.8809

Results. We compare the baselines with our encoder model incorporating DICE (Figure 4(a)), using a GNN depth configuration of $(d_D, d_p, d_s) = (2, 0, 2)$. Figure 6 shows the validation performance curves of all encoder-decoder models during training. Final performance comparisons for each downstream task are summarized in Table 2. Each model was trained using three different random seeds, and we report the average performance across these models. Both Figure 6 and Table 2 demonstrate that our model outperforms all baselines across the downstream tasks.

4.3 Ablation Studies

(1) $\mathcal{L}_{\text{SimSiam}}$ vs. $\mathcal{L}_{\text{NT-Xent}}$ vs. $\mathcal{L}_{\text{DICE}}$. We also applied GNNs pretrained with $\mathcal{L}_{\text{SimSiam}}$ and $\mathcal{L}_{\text{NT-Xent}}$ in the encoder of Figure 4 and evaluated them on our downstream tasks. As shown in Table 3, both methods improve performance over the baselines in tasks 2 and 3.

(2) **DICE vs. Diverse GNNs.** We also evaluated the effect of pretraining different GNN architectures, with results provided in Table 3. GCN [48], GraphSAGE [49], GAT [50], and GIN [51] were pretrained using $\mathcal{L}_{\text{DICE}}$ (see Appendix C). Pretraining diverse GNNs also leads to performance gains.

(3) **Training from Scratch vs. Applying DICE.** We compared two models: one trained from scratch and the other incorporating DICE. With both models following the architecture shown in Figure 4, the encoder remains fixed across all downstream tasks, and the decoder varies by task but is consistent across models within each task. d_D is set to 0 for models without DICE, and d_D is set to 2 for models

Table 3: **Results of ablation studies (1) and (2).** Bolded values outperform all baselines in Table 2. Each pretrained GNN serves as part of the encoder in Figure 4, with $(d_D, d_p, d_s) = (2, 0, 2)$.

Pretraining Loss	Pretrained GNN	Task 1 (%)	Task 2 (R^2)		Task 3 (R^2)				
		Accuracy	Rise Delay	Fall Delay	Power	V _{offset}	CMRR	Gain	PSRR
$\mathcal{L}_{\text{SimSiam}}$	DICE	85.88	0.9427	0.8044	0.9911	0.9951	0.9353	0.9762	0.8768
$\mathcal{L}_{\text{NT-Xent}}$	DICE	76.73	0.9492	0.7706	0.9898	0.9947	0.9328	0.9757	0.8720
$\mathcal{L}_{\text{DICE}}$	GCN	85.92	0.9113	0.7467	0.9909	0.9940	0.9314	0.9743	0.8662
$\mathcal{L}_{\text{DICE}}$	GraphSAGE	79.35	0.8975	0.7786	0.9899	0.9947	0.9346	0.9756	0.8702
$\mathcal{L}_{\text{DICE}}$	GAT	84.32	0.9314	0.7981	0.9862	0.9867	0.9130	0.9624	0.8004
$\mathcal{L}_{\text{DICE}}$	GIN	93.88	0.9414	0.7931	0.9915	0.9947	0.9336	0.9757	0.8788
$\mathcal{L}_{\text{DICE}}$	DICE	93.94	0.9468	0.8085	0.9916	0.9950	0.9374	0.9769	0.8809

Table 4: **Results on ablation study (3).** The final performance values are bolded if they surpass the value of the counterparts, which indicates models having the same (d_p, d_s) values but different d_D values. Per task metric, the best performance values across all models are marked with underlines.

(d_D, d_p, d_s)		Task 1 (%)	Task 2 (R^2)		Task 3 (R^2)				
		Accuracy	d_{rise}	d_{fall}	Power	V_{offset}	CMRR	Gain	PSRR
MPNN without DICE	(0, 0, 0)	76.91	0.7541	0.5456	0.9651	0.8261	0.4794	0.3754	0.2798
	(0, 0, 1)	74.84	0.7706	0.5486	0.9718	0.8976	0.6384	0.6441	0.5024
	(0, 1, 0)	77.90	0.7666	0.5405	0.9636	0.8825	0.6019	0.6013	0.4501
	(0, 1, 1)	84.68	0.9404	0.8124	0.9827	0.9876	0.9125	0.9633	0.8064
	(0, 0, 2)	88.57	0.9229	0.7790	0.9745	0.9863	0.9091	0.9605	0.7929
	(0, 2, 0)	86.28	0.9237	0.7618	0.9866	0.9867	0.9066	0.9610	0.7900
MPNN with DICE	(2, 0, 0)	92.13	0.9352	0.7885	0.9769	0.9820	0.8865	0.9402	0.7535
	(2, 0, 1)	93.41	0.9310	0.7805	0.9904	0.9876	0.9167	0.9659	0.8132
	(2, 1, 0)	93.43	0.9294	0.7820	0.9849	0.9868	0.9147	0.9634	0.8061
	(2, 1, 1)	<u>94.54</u>	<u>0.9417</u>	0.7873	0.9847	0.9872	0.9147	0.9627	0.8061
	(2, 0, 2)	94.01	0.9231	0.8134	0.9918	0.9950	0.9371	0.9769	0.8776
	(2, 2, 0)	93.98	0.9402	0.7851	0.9876	0.9874	0.9165	0.9641	0.8055

with DICE. The number of trainable parameters is kept constant across models with identical (d_p, d_s) values. Table 4 presents the experimental results, and all bolded and underlined values correspond to the MPNN model incorporating DICE, except for a few cases when $(d_p, d_s) = (1, 1)$.

5 Discussion

Based on the results from both baseline and ablation studies, we highlight four key observations:

First, applying pretrained GNNs significantly improves performance compared to training from scratch. As shown in Table 2, our model using DICE outperforms ParaGraph with up to a 15.19% average improvement on Task 3, and exceeds DeepGen_u with up to a 44.24% average improvement on the same task. Ablation study (3) further confirms the effectiveness of DICE, achieving up to a 19.79% improvement on Task 1 when $(d_p, d_s) = (0, 0)$.

Second, graph contrastive pretraining is substantially more effective than the supervised pretraining method proposed in [47]. In Table 2, our model outperforms DeepGen_p by 10.97% on Task 1 and achieves a 9.67% average improvement on Task 2. These results underscore the advantage of unsupervised learning over supervised approaches for task-agnostic pretraining.

Third, combining negative augmentation with similarity minimization improves performance. In Task 1 of ablation study (1), our model incorporating DICE outperforms the one pretrained with $\mathcal{L}_{\text{SimSiam}}$ by 9.39%, and exceeds the model pretrained with $\mathcal{L}_{\text{NT-Xent}}$ by 22.43%. The results follow the conclusion in [34], arguing that similarity minimization is not necessary in contrastive learning. However, results also show that using $\mathcal{L}_{\text{DICE}}$ with our negative augmentation outperforms the approach in [34].

Fourth, the proposed pretraining framework generalizes well across diverse GNN architectures. In Task 3 of ablation study (2), all four pretrained models outperform ParaGraph by an average of 14.09% and exceed DeepGen_u by 42.87%. These results demonstrate the robustness of our method.

6 Conclusion

In this work, we propose DICE, a pretrained GNN model designed for general graph-level prediction tasks in device-level integrated circuits. Our primary contribution is to highlight the importance of pretraining graph models specifically on device-level circuits, and we introduce the first graph contrastive learning framework to address this challenge. We argue that device-level representations offer a more general and flexible abstraction for integrated circuits than logic gate-level representations, as they support both analog and digital designs. Another contribution is the introduction of two novel data augmentation techniques that address the scarcity of circuit data. Additionally, we propose a new device-level circuit benchmark, where all three tasks require handling multiple circuit topologies. Experimental results including the ablation studies show that incorporating DICE leads to significant improvements across all three downstream tasks. We view this work as a step toward building general-purpose models that unify analog and digital circuit design.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [3] Jiasen Lu, Dhruv Batra, Devi Parikh, and Stefan Lee. Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. *Advances in neural information processing systems*, 32, 2019.
- [4] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [5] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*, 4(12):1256–1264, 2022.
- [6] Xiangxiang Zeng, Hongxin Xiang, Linhui Yu, Jianmin Wang, Kenli Li, Ruth Nussinov, and Feixiong Cheng. Accurate prediction of molecular properties and drug targets using a self-supervised image representation learning framework. *Nature Machine Intelligence*, 4(11):1004–1016, 2022.
- [7] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep graph infomax. *arXiv preprint arXiv:1809.10341*, 2018.
- [8] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. *arXiv preprint arXiv:1908.01000*, 2019.
- [9] Shaika Chowdhury, Chenwei Zhang, Philip S Yu, and Yuan Luo. Mixed pooling multi-view attention autoencoder for representation learning in healthcare. *arXiv preprint arXiv:1910.06456*, 2019.
- [10] Michelle M Li, Kexin Huang, and Marinka Zitnik. Graph representation learning in biomedicine and healthcare. *Nature Biomedical Engineering*, 6(12):1353–1369, 2022.
- [11] Tomohisa Seki, Yoshimasa Kawazoe, and Kazuhiko Ohe. Clinical feature vector generation using unsupervised graph representation learning from heterogeneous medical records. In *AMIA Annual Symposium Proceedings*, volume 2023, page 618, 2024.
- [12] Shengchao Liu, Mehmet F Demirel, and Yingyu Liang. N-gram graph: Simple unsupervised representation for graphs, with applications to molecules. *Advances in neural information processing systems*, 32, 2019.
- [13] Yuyang Wang, Jianren Wang, Zhonglin Cao, and Amir Barati Farimani. Molecular contrastive learning of representations via graph neural networks. *Nature Machine Intelligence*, 4(3):279–287, 2022.
- [14] Tianyu Zhang, Yuxiang Ren, Chengbin Hou, Hairong Lv, and Xuegong Zhang. Molecular graph representation learning integrating large language models with domain-specific small models. In *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 2928–2935. IEEE, 2024.
- [15] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Ru Huang, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, et al. The dawn of ai-native eda: Opportunities and challenges of large circuit models. *arXiv preprint arXiv:2403.07257*, 2024.

- [16] Ziyi Wang, Chen Bai, Zhuolun He, Guangliang Zhang, Qiang Xu, Tsung-Yi Ho, Bei Yu, and Yu Huang. Functionality matters in netlist representation learning. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 61–66, 2022.
- [17] Ziyi Wang, Chen Bai, Zhuolun He, Guangliang Zhang, Qiang Xu, Tsung-Yi Ho, Yu Huang, and Bei Yu. Fgnn2: A powerful pre-training framework for learning the logic functionality of circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [18] Zhengyuan Shi, Hongyang Pan, Sadaf Khan, Min Li, Yi Liu, Junhua Huang, Hui-Ling Zhen, Mingxuan Yuan, Zhufei Chu, and Qiang Xu. Deepgate2: Functionality-aware circuit representation learning. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2023.
- [19] Zhengyuan Shi, Ziyang Zheng, Sadaf Khan, Jianyuan Zhong, Min Li, and Qiang Xu. Deepgate3: Towards scalable circuit representation learning. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.
- [20] Ziyang Zheng, Shan Huang, Jianyuan Zhong, Zhengyuan Shi, Guohao Dai, Ningyi Xu, and Qiang Xu. Deepgate4: Efficient and effective representation learning for circuit design at scale. *arXiv preprint arXiv:2502.01681*, 2025.
- [21] Yanqiao Zhu, Yichen Xu, Qiang Liu, and Shu Wu. An empirical study of graph contrastive learning. *arXiv preprint arXiv:2109.01116*, 2021.
- [22] Wei Ju, Yifan Wang, Yifang Qin, Zhengyang Mao, Zhiping Xiao, Junyu Luo, Junwei Yang, Yiyang Gu, Dongjie Wang, Qingqing Long, et al. Towards graph contrastive learning: A survey and beyond. *arXiv preprint arXiv:2405.11868*, 2024.
- [23] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. Graph contrastive learning with augmentations. *Advances in neural information processing systems*, 33:5812–5823, 2020.
- [24] Dongkuan Xu, Wei Cheng, Dongsheng Luo, Haifeng Chen, and Xiang Zhang. Infogcl: Information-aware graph contrastive learning. *Advances in Neural Information Processing Systems*, 34:30414–30425, 2021.
- [25] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. Graph contrastive learning with adaptive augmentation. In *Proceedings of the web conference 2021*, pages 2069–2080, 2021.
- [26] Clifford Wolf, Johann Glaser, and Johannes Kepler. Yosys-a free verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, volume 97, 2013.
- [27] Robert Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*, pages 24–40. Springer, 2010.
- [28] Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 667–672, 2022.
- [29] Hanrui Wang, Kuan Wang, Jiacheng Yang, Linxiao Shen, Nan Sun, Hae-Seung Lee, and Song Han. Gcn-rl circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.
- [30] Keertana Settaluri, Ameer Haj-Ali, Qijing Huang, Kourosh Hakhmaneshi, and Borivoje Nikolic. Autockt: Deep reinforcement learning of analog circuit designs. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 490–495. IEEE, 2020.
- [31] Ahmet F Budak, Prateek Bhansali, Bo Liu, Nan Sun, David Z Pan, and Chandramouli V Kashyap. Dnn-opt: An rl inspired optimization for analog circuit sizing using deep neural networks. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 1219–1224. IEEE, 2021.

- [32] Souradip Poddar, Youngmin Oh, Yao Lai, Hanqing Zhu, Bosun Hwang, and David Z Pan. Insight: Universal neural simulator for analog circuits harnessing autoregressive transformers. *arXiv preprint arXiv:2407.07346*, 2024.
- [33] Zihu Wang, Karthik NS Somayaji, and Peng Li. Learn-by-compare: Analog performance prediction using contrastive regression with design knowledge. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6, 2024.
- [34] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PMLR, 2020.
- [35] Xinlei Chen and Kaiming He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15750–15758, 2021.
- [36] Alan Mishchenko, Satrajit Chatterjee, and Robert Brayton. Dag-aware aig rewriting a fresh look at combinational logic synthesis. In *Proceedings of the 43rd annual Design Automation Conference*, pages 532–535, 2006.
- [37] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, et al. Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6, 2024.
- [38] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8. IEEE, 2023.
- [39] Wenji Fang, Shang Liu, Jing Wang, and Zhiyao Xie. Circuitfusion: multimodal circuit representation learning for agile chip design. *arXiv preprint arXiv:2505.02168*, 2025.
- [40] Kourosh Hakhamaneshi, Marcel Nassar, Mariano Phielipp, Pieter Abbeel, and Vladimir Stojanovic. Pretraining graph neural networks for few-shot analog circuit modeling and design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(7): 2163–2173, 2022.
- [41] Ali Deeb, Abdalrahman Ibrahim, Mohamed Salem, Joachim Pichler, Sergii Tkachov, Anjeza Karaj, Fadi Al Machot, and Kyamakya Kyandoghere. A robust automated analog circuits classification involving a graph neural network and a novel data augmentation strategy. *Sensors*, 23(6):2989, 2023.
- [42] Ali Deeb, Mohamed Salem, Abdalrahman Ibrahim, Joachim Pichler, Sergii Tkachov, Karaj Anjeza, Fadi Al Machot, and Kyandoghere Kyamakya. A graph attention network based system for robust analog circuits’ structure recognition involving a novel data augmentation technique. *IEEE Access*, 12:29308–29344, 2024.
- [43] Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z Pan, and Ping Luo. Analogcoder: Analog circuit design via training-free code generation. *arXiv preprint arXiv:2405.14918*, 2024.
- [44] Deepak Vungarala, Sakila Alam, Arnob Ghosh, and Shaahin Angizi. Spicepilot: Navigating spice code generation and simulation with ai guidance. *arXiv preprint arXiv:2410.20553*, 2024.
- [45] Jitendra Bhandari, Vineet Bhat, Yuheng He, Siddharth Garg, Hamed Rahmani, and Ramesh Karri. Auto-spice: Leveraging llms for dataset creation via automated spice netlist extraction from analog circuit diagrams. *arXiv preprint arXiv:2411.14299*, 2024.
- [46] Haoxing Ren, George F Kokai, Walker J Turner, and Ting-Sheng Ku. Paragraph: Layout parasitics and device parameter prediction using graph neural networks. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

- [47] Kourosh Hakhamaneshi, Marcel Nassar, Mariano Phielipp, Pieter Abbeel, and Vladimir Stojanovic. Pretraining graph neural networks for few-shot analog circuit modeling and design. *IEEE Trans. CAD of Integrated Circuits and Systems (TCAD)*, 2023.
- [48] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [49] Will Hamilton, Zhitaoying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [50] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.
- [51] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*, 2018.
- [52] Fuping Li, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. Noception: A fast ppa prediction framework for network-on-chips using graph neural network. In *Design, Automation & Test in Europe (DATE)*, 2022.
- [53] Wenji Fang, Yao Lu, Shang Liu, Qijun Zhang, Ceyu Xu, Lisa Wu Wills, Hongce Zhang, and Zhiyao Xie. Masterrtl: A pre-synthesis ppa estimation framework for any rtl design. *arXiv preprint arXiv:2311.08441*, 2023.
- [54] Yufan Du, Zizheng Guo, Xun Jiang, Zhuomin Chai, Yuxiang Zhao, Yibo Lin, Runsheng Wang, and Ru Huang. PowPredICT: Cross-stage power prediction with circuit-transformation-aware learning. In *Proceedings of the 61st Design Automation Conference (DAC)*, 2024.
- [55] Zizheng Guo, Mingjie Liu, Jiaqi Gu, Shuhan Zhang, David Z. Pan, and Yibo Lin. A timing engine inspired graph neural network model for pre-routing slack prediction. In *Proceedings of the 59th Design Automation Conference (DAC)*, 2022.
- [56] Sanjay Gandham, Joe Walston, Sourav Samanta, Lingxiang Yin, Hao Zheng, Mingjie Lin, and Stelios Diamantidis. CircuitSeer: RTL post-pnr delay prediction via coupling functional and structural representation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- [57] Bowen Wang, Guibao Shen, Dong Li, Jianye Hao, Wulong Liu, Yu Huang, Hongzhong Wu, Yibo Lin, Guangyong Chen, and Pheng-Ann Heng. LHNN: Lattice hypergraph neural network for vlsi congestion prediction. In *Proceedings of the 59th Design Automation Conference (DAC)*, 2022.
- [58] Yuxiang Zhao, Zhuomin Chai, Xun Jiang, Yibo Lin, Runsheng Wang, and Ru Huang. PDNNet: PDN-aware gnn-cnn heterogeneous network for dynamic ir drop prediction. *arXiv preprint arXiv:2403.18569*, 2024.
- [59] Lilas Alrahis, Abhrajit Sengupta, Johann Knechtel, Satwik Patnaik, Hani Saleh, Baker Mohammad, Mahmoud Al-Qutayri, and Ozgur Sinanoglu. GNN-RE: Graph neural networks for reverse engineering of gate-level netlists. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 41(8):2435–2448, 2022.
- [60] Tim Bücher, Lilas Alrahis, Guilherme Paim, Sergio Bampi, Ozgur Sinanoglu, and Hussam Amrouch. Appggn: Approximation-aware functional reverse engineering using graph neural networks. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2022.
- [61] Aditya Hemant Shahane, Swapna Manjiri, Ankesh Jain, and Sandeep Kumar. Graph of circuits with gnn for exploring the optimal design space. In *NeurIPS*, 2023.
- [62] Zhengfeng Wu and Ioannis Savidis. Circuit-gnn: A graph neural network for transistor-level modeling of analog circuit hierarchies. In *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2023.

- [63] Ahmed K. Khamis and Mohammed Agamy. Circuit topology aware gnn-based multi-variable model for dc-dc converters dynamics prediction in ccm and dcm. *Neural Computing and Applications*, 36:20807–20822, 2024.
- [64] Mingjie Liu, Walker J. Turner, George F. Kokai, Brucek Khailany, David Z. Pan, and Haoxing Ren. Parasitic-aware analog circuit sizing with graph neural networks and bayesian optimization. In *Proc. Design Automation and Test in Europe (DATE)*, 2021.
- [65] Hao Chen, Keren Zhu, Mingjie Liu, Xiyuan Tang, Nan Sun, and David Z. Pan. Universal symmetry constraint extraction for analog and mixed-signal circuits with graph neural networks. In *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2021.
- [66] Hyunsu Chae, Hao Yu, Sensen Li, and David Z Pan. Pulserf: Physics augmented ml modeling and synthesis for high-frequency rfc design. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.
- [67] Kishor Kunal, Tonmoy Dhar, Meghna Madhusudan, Jitesh Poojary, Arvind K Sharma, Wenbin Xu, Steven M Burns, Jiang Hu, Ramesh Harjani, and Sachin S Sapatnekar. Gnn-based hierarchical annotation for analog circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(9):2801–2814, 2023.
- [68] Qi Xu, Lijie Wang, Jing Wang, Lin Cheng, Song Chen, and Yi Kang. Graph attention-based symmetry constraint extraction for analog circuits. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [69] Zehao Dong, Weidong Cao, Muhan Zhang, Dacheng Tao, Yixin Chen, and Xuan Zhang. Cktgcn: Circuit graph neural network for electronic design automation. *arXiv preprint arXiv:2308.16406*, 2023.
- [70] Jintao Li, Haochang Zhi, Ruiyu Lyu, Wangzhen Li, Zhaori Bi, Keren Zhu, Yanhan Zeng, Weiwei Shan, Changhao Yan, Fan Yang, et al. Analoggym: An open and practical testing suite for analog circuit synthesis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.
- [71] Asal Mehradfar, Xuzhe Zhao, Yue Niu, Sara Babakniya, Mahdi Alesheikh, Hamidreza Aghasi, and Salman Avestimehr. Aicircuit: A multi-level dataset and benchmark for ai-driven analog integrated circuit design. *arXiv preprint arXiv:2407.18272*, 2024.
- [72] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. Deepergcn: All you need to train deeper gcns. *arXiv preprint arXiv:2006.07739*, 2020.
- [73] Jian Gao, Weidong Cao, Junyi Yang, and Xuan Zhang. Analoggenie: A generative engine for automatic discovery of analog circuit topologies. *arXiv preprint arXiv:2503.00205*, 2025.
- [74] Shih-Hsin Wang, Yuhao Huang, Justin M. Baker, Yuan-En Sun, Qi Tang, and Bao Wang. A theoretically-principled sparse, connected, and rigid graph representation of molecules. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=0Ivg3MqWX2>.
- [75] Weidong Cao, Mouhacine Benosman, Xuan Zhang, and Rui Ma. Domain knowledge-infused deep learning for automated analog/radio-frequency circuit parameter optimization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 1015–1020, 2022.
- [76] Jian Gao, Weidong Cao, and Xuan Zhang. Rose: Robust analog circuit parameter optimization with sampling-efficient reinforcement learning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.
- [77] Weidong Cao, Jian Gao, Tianrui Ma, Rui Ma, Mouhacine Benosman, and Xuan Zhang. Rose-opt: Robust and efficient analog circuit parameter optimization with knowledge-infused reinforcement learning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

A Related Works

A.1 Graph-level Prediction Tasks in EDA

Predicting properties of the entire circuit can significantly reduce design time, as simulations are often time-consuming. To address this, prior work on digital circuits has focused on training models to predict power, performance, and area (PPA) [52–54], timing violations [55, 56], routing congestions [57, 58], and functional similarities between digital circuits [59, 60]. For analog circuits, previous studies have trained models to predict performance metrics such as gain, bandwidth, and power consumption [46, 47, 61–66], and also trained models to identify functional similarities between analog circuit blocks or transistor groups [41, 42, 67, 68]. In this work, we show that our pretrained model DICE is effective in solving similar graph-level prediction tasks.

A.2 Benchmarks for Device-level Circuits

Benchmarks for graph-level prediction tasks exist for both analog and digital circuits; however, digital circuit benchmarks are unsuitable for device-level circuits because their graph representations are based on logic gates rather than device components. Analog circuit benchmarks also pose challenges when used to evaluate graph learning methods at the device level. The Open Circuit Benchmark (OCB)[69] provides various operational amplifier topologies, but it is not suitable for device-level circuit evaluation since its circuit graphs do not use transistors as fundamental components. Analoggym[70] and AICircuit [71] offer multiple circuit topologies, but the associated tasks focus on device parameter optimization, which are inherently node-level tasks. While [32, 33] address graph-level prediction, each task in their benchmarks involves only a single circuit topology. These limitations underscore the need for new graph-level prediction benchmarks that support multiple circuit topologies and represent circuits using device-level components such as transistors.

A.3 Converting Device-Level Circuits into Graphs

There exist various approaches for converting device-level circuits into graphs. A key challenge lies in how to represent transistor devices as graph components. The work in [46] models both devices and voltage nets as graph nodes and distinguishes connection types using multiple edge types. They adopt a heterogeneous GNN (ParaGraph) with an attention mechanism, and omit power and ground voltage nets from the graph as nodes. The studies in [40, 47] pretrain the GNN (DeepGen) proposed in [72], using a supervised node-level task: DC voltage prediction. Their graph construction does not differentiate edge types and instead separates device terminals into individual nodes, disjoint from their corresponding device nodes.

Recent works [40, 47, 73] follow a similar modeling approach, treating each device pin as a separate node. However, such representations can lead to inefficient message passing when applied in Message Passing Neural Networks (MPNNs). For example, the edge from a gate node to a transistor device should be directional, not bidirectional. This is because the voltage signal at the gate controls the current flow through the transistor, but the transistor exerts minimal influence on the gate voltage. To address this limitation, we introduce a new mapping method that maintains directional semantics with edge features and minimizes redundant message passing.

A.4 Including Device Parameters in the Pretraining Dataset

Training a general-purpose model requires constructing a comprehensive dataset. A straightforward approach to building a large-scale circuit dataset is to enumerate various device parameter configurations [69, 71]. However, this method may fail to capture the full breadth of circuit knowledge. Consider a circuit with twenty devices, each with three possible parameter values—this results in 3^{20} potential combinations, yielding an enormous dataset from a single topology. Yet, constraining each device to only three parameter values is insufficient to represent the true complexity of circuit behavior, as meaningful parameters vary widely across different tasks and technologies. Moreover, capturing structural diversity requires incorporating multiple circuit topologies, which further expands the dataset and introduces additional inefficiencies. Therefore, instead of relying on device parameter variations, we employ data augmentation to generate a sufficiently large pretraining dataset.

B Details of Data Augmentation

B.1 Positive Data Augmentation Details

We explain why positive data augmentation leads to the preservation of graph-level functionality. Consider two resistors with resistance values r_1 and r_2 . When connected in series, these resistors can be equivalently replaced by a single resistor with resistance value of $r_1 + r_2$. Similarly, for parallel connections, they can be replaced by a resistor with resistance value of $\frac{r_1 r_2}{r_1 + r_2}$. This principle applies analogously to all other circuit components. Since device parameters do not affect structural circuit properties, we can consolidate series/parallel-connected devices into single components. Consequently, we can make positive samples consistently converge toward unified topological structures (in our case, 55 initial circuits) through these replacements.

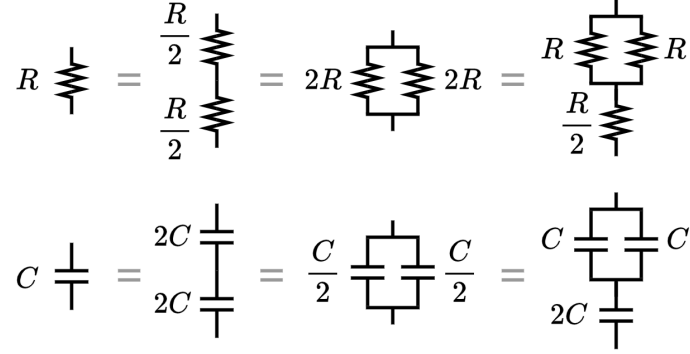


Figure 7: Equivalent subgraphs generated through positive data augmentation. Different subgraph topologies indicate the same local functionality, and this leads to equal graph-level functions if all other structures are the same.

B.2 Negative Data Augmentation Details

We explain why negative data augmentation leads to the perturbation of graph-level functionality. For passive elements, we consider their impedance values at both DC (zero frequency) and the high-frequency limit, and modify them to have the inverted impedance characteristics at both extremes. For current sources, we replace them with passive elements, which are equivalent to converting the energy supplier to an energy dissipator. Finally, for transistors, we consider switching behaviors based on gate voltage inputs and connect the counterpart transistors in both parallel and series. Since either impedance characteristics, energy consumption, or switching behavior is inverted, the graph-level functionality is modified with our proposing negative data augmentation.

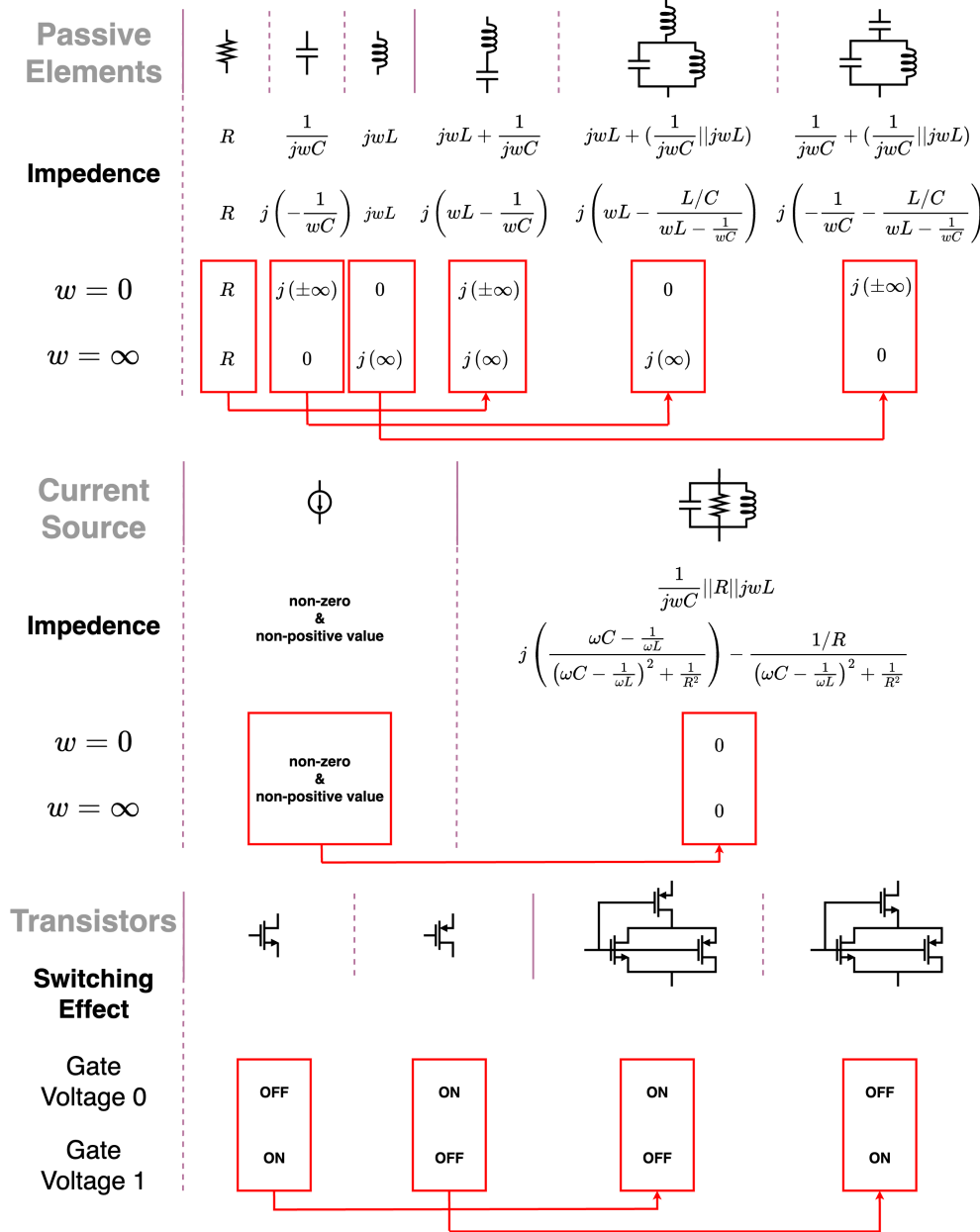


Figure 8: Conversion of characteristics through negative data augmentation.

C GNN update rules

C.1 DICE

The update rule of DICE follows the Graph Isomorphism Network (GIN) with edge feature updates.

$$\mathbf{h}_v^{(1)} = MLP_{\theta_0}(\mathbf{h}_v^{(0)}), \quad \mathbf{e}_{u \rightarrow v}^{(1)} = MLP_{\theta_1}(\mathbf{e}_{u \rightarrow v}^{(0)}) \quad (4)$$

$$\mathbf{m}_v^{(k)} = \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k)} \cdot \mathbf{e}_{u \rightarrow v}^{(k)},$$

$$\mathbf{h}_v^{(k+1)} = MLP_{\theta_2^{(k)}}((1 + \phi_h^{(k)}) \cdot \mathbf{h}_v^{(k)} + \mathbf{m}_v^{(k)}), \quad (5)$$

$$\mathbf{e}_{u \rightarrow v}^{(k+1)} = MLP_{\theta_3^{(k)}}((1 + \phi_e^{(k)}) \cdot \mathbf{e}_{u \rightarrow v}^{(k)} + \mathbf{m}_u^{(k)} - \mathbf{m}_v^{(k)})$$

$$\mathbf{g}_{(V,E)} = \sum_{v \in V} \mathbf{h}_v^{(l+1)} + \sum_{u \rightarrow v \in E} \mathbf{e}_{u \rightarrow v}^{(l+1)} \quad (6)$$

Eqs. (4)–(6) illustrates the node and edge feature update rules of DICE, where $\theta_0, \theta_1, \theta_2^{(k)}, \theta_3^{(k)}, \phi_h^{(k)}$, and $\phi_e^{(k)}$ are the training parameters. Batch normalization and dropout layers are included between linear layers. Eq. (4) matches the initial dimensions of node and edge features (9 and 5) to the hidden dimension value, and $\mathbf{h}_u^{(0)}, \mathbf{e}_{v \rightarrow u}^{(0)}$ are the initial one-hot encoding of node and edge types. Eq. (5) explains the message passing rule, where $k = 1, \dots, l$ and $l (= 2)$ is the depth of the GNN. The graph-level feature $\mathbf{g}_{(V,E)}$ is calculated in Eq. (6), by summing all the updated node and edge features within each graph $G = (V, E)$.

C.2 GCN

The update rule of the Graph Convolutional Network (GCN) used in this work follows a normalized aggregation mechanism:

$$\mathbf{m}_v^{(k)} = \sum_{u \in \mathcal{N}(v)} \frac{1}{\sqrt{d_v d_u}} \cdot \mathbf{h}_u^{(k)} \quad (7)$$

$$\mathbf{h}_v^{(k+1)} = MLP_{\theta^{(k)}}(\mathbf{m}_v^{(k)})$$

Here, d_v and d_u denote the degrees of nodes v and u , respectively. The GCN layer performs degree-normalized aggregation followed by a learnable transformation of the aggregated features. Although edge features are not explicitly involved in the message-passing process for GCN, Eq. (4) is still applied at the initial layer to project both node and edge features into a common hidden space. Eq. (6) is used to compute the graph-level feature by summing the final node and edge embeddings.

C.3 GraphSAGE

GraphSAGE in our work performs aggregation by concatenating the central node’s embedding with the mean of its neighbors’ embeddings:

$$\mathbf{m}_v^{(k)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k)} \quad (8)$$

$$\mathbf{h}_v^{(k+1)} = MLP_{\theta^{(k)}}([\mathbf{h}_v^{(k)} \parallel \mathbf{m}_v^{(k)}])$$

Aggregation is performed through mean pooling, and node features are updated by concatenating the aggregated messages with the node’s current embedding. Although edge features do not contribute to the message-passing step, Eq. (4) is used to initialize the edge feature embeddings. The final graph-level feature is computed via Eq. (6), which includes both node and edge representations.

C.4 GAT

The Graph Attention Network (GAT) used in our work computes attention coefficients for each edge to weigh the contributions of neighboring nodes:

$$\begin{aligned}\alpha_{u \rightarrow v}^{(k)} &= \text{softmax}_{u \in \mathcal{N}(v)} \left(\mathbf{h}_u^{(k)} \cdot \mathbf{h}_v^{(k)} \right) \\ \mathbf{m}_v^{(k)} &= \sum_{u \in \mathcal{N}(v)} \alpha_{u \rightarrow v}^{(k)} \cdot \mathbf{h}_u^{(k)} \\ \mathbf{h}_v^{(k+1)} &= \mathbf{h}_v^{(k)} + \mathbf{m}_v^{(k)}\end{aligned}\tag{9}$$

The attention mechanism allows the model to learn the relative importance of neighboring nodes based on their feature similarity. While edge features are not updated during message passing in GAT, they are initially transformed using Eq. (4). The final graph-level representation is computed by summing both the node and edge embeddings as described in Eq. (6).

C.5 GIN

The update rule of the Graph Isomorphism Network (GIN) uses sum aggregation followed by a learnable MLP:

$$\begin{aligned}\mathbf{m}_v^{(k)} &= \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k)} \\ \mathbf{h}_v^{(k+1)} &= \text{MLP}_{\theta^{(k)}} \left((1 + \epsilon^{(k)}) \cdot \mathbf{h}_v^{(k)} + \mathbf{m}_v^{(k)} \right)\end{aligned}\tag{10}$$

Here, $\epsilon^{(k)}$ is a learnable scalar, initialized as zero. GIN is known for its strong expressive power in distinguishing graph structures. Although the edge features are not used in message passing for GIN, they are projected to the hidden space using Eq. (4). Both node and edge embeddings are summed to form the graph-level representation, as defined in Eq. (6).

D Details of the Experiments

D.1 Pretraining

D.1.1 Dataset

Data augmentation is used exclusively for generating the pretraining datasets and is performed independently for the pretraining training and test sets. The pretraining dataset comprises 55 initial circuit topologies, covering both analog and digital designs. For the training set, 40 topologies are each augmented 4,000 times (2,000 positive and 2,000 negative), resulting in a total of 160,000 graphs. For testing with t-SNE visualization, the remaining 15 topologies are each augmented 500 times using only positive augmentation, yielding 7,500 graphs. For testing cosine similarity values, the same 15 topologies are each augmented 500 times (250 positive and 250 negative), also resulting in 7,500 graphs. We did not use negative augmentation for t-SNE visualization since samples with negative augmentation cannot be categorized and distracts the visualization results.

D.1.2 Contrastive Learning

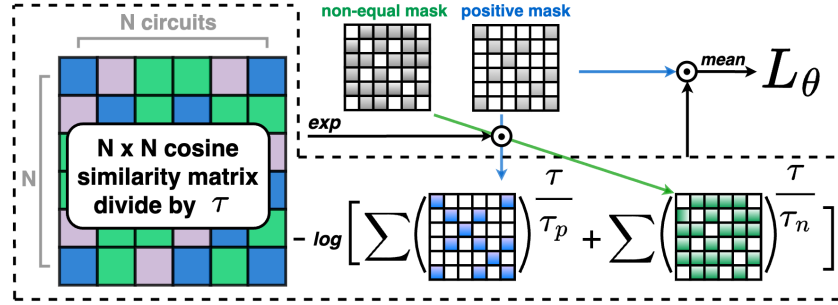


Figure 9: **Masking operation.** $\odot$ indicates the hadamard product (element-wise multiplication).

Masking is our key source of rapid pretraining, with the cost of GPU memory. Algorithm. 1 provides the pseudocode and Fig. 9 visualize the details of the the proposed masking for contrastive learning. For each batch, we first matched the number of graphs for every circuit types. For example, consider the case when the number of graphs originated from the inverter circuit is 5. If this is the minimum number among all other circuit types in the batch, then 5 circuits are sampled for every circuit types and form a new batch. Then, the cosine similarity matrix is formed with graph-level features in the batch with masks indicating positive and non-equal. Element-wise products between the similarity matrix and the masks filter out the unnecessary parts in parallel. Multiple positive pairs are considered simultaneously with this operation, thereby significantly reducing the training speed with the cost of GPU memory.

Algorithm 1 Graph-level Contrastive Learning

Input: augmented dataset (χ), epoch (T), temperature coefficients (τ_p, τ, τ_n), learning rate (η)

Output: pretrained model parameter (θ_T)

Initialize: model parameter θ_0

```

1: for  $t = 0, 1, 2, \dots, T - 1$  do
2:   for batch  $B$  in  $\chi$  do
3:      $m \leftarrow$  min number covering all circuit types in  $B$ 
4:      $B_m \leftarrow$  batch  $\in \mathbb{R}^L$  having  $m$  graphs each for every circuit types (maintaining circuit diversity)
5:      $g \leftarrow GNN_{\theta_t}(B_m)$ : graph-level features  $\in \mathbb{R}^{L \times h}$ 
6:      $S \leftarrow gg^T$ : cosine similarity matrix  $\in \mathbb{R}^{L \times L}$ 
7:      $M_+, M_\neq \leftarrow$  Mask matrices  $\in \mathbb{R}^{L \times L}$  for positive and non-equal pairs
9:      $L_{\theta_t} \leftarrow \sum (M_+ * [S/\tau - \log(\sum M_+ * e^{S/\tau_p} + \sum M_\neq * e^{S/\tau_n})]) / \sum M_+$ 
10:     $\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta_t} L_{\theta_t}$ 
11:   end for
12: end for

```

D.2 Downstream Tasks

D.2.1 Task 1: Circuit Similarity Prediction

Setup. The model predicts the relative similarities between three circuits, including both analog and digital topologies. The objective is to maximize prediction accuracy (%) based on true similarity, which is determined by the number of shared labels. A total of 6 labels include analog, digital, delay lines, amplifiers, logic gates, and oscillators.

Given a target circuit and two comparison circuits (total of three), the model outputs a three-dimensional probability distribution: whether the first circuit is more similar, the second circuit is more similar, or the two circuits are equally similar to the target circuit. In each training step, one target circuit from the training dataset is sampled, and all permuted pairs across the N number of circuits form a tensor of size $(N \times (N - 1) = 2950, 3 \times \text{graph feature dimension})$ for the training model. Based on the number of shared labels, the logits indicating similarity comparisons are computed. Using these logits and the model output, the cross-entropy loss is minimized.

Dataset. The training dataset consists of 50 circuits, including the 40 topologies from the pretraining dataset. The test dataset contains 5 additional circuits, resulting in a total of 55 circuits.

D.2.2 Task 2: Delay Prediction

Setup. The model predicts simulation results for five delay line circuits, which operate on digital signals. Each simulation records two values: the time difference between the rising edges and the falling edges of the input and output clock signals. The objective is to maximize the coefficient of determination (R^2) across all simulations, which measures how well the model’s predictions match the true simulation results.

Dataset. For simulation, we used NGSPICE with the BSIM4 45nm technology, which serve as the open-source circuit simulator and technology file, respectively. The delay line circuits used in Task 2 are not included in the pretraining dataset. The simulation result dataset contains 45,000 device parameter combinations for each circuit topology. We split the dataset into training, validation, and test sets with a ratio of 8:1:1. The output dimension of the decoder model is 2.

D.2.3 Task 3: OPAMP Metric Prediction

Setup. The model predicts simulation results for five operational amplifier (op-amp) circuits, which are analog circuits. Each simulation records five metrics: power, DC voltage offset (V_{offset}), common-mode rejection ratio (CMRR), gain, and power supply rejection ratio (PSRR). The objective is to maximize the coefficient of determination (R^2) across all simulations, which evaluates how well the model predictions align with the true simulation results.

Dataset. NGSPICE with the BSIM4 45nm technology file is also used for Task 3. The op-amp circuit topologies used in Task 3 are included in the pretraining dataset, but the simulation results are newly generated for this task. The simulation dataset contains 60,000 device parameter combinations for each topology. The circuit and testbench files are sourced from [70]. The dataset is split into training, validation, and test sets using an 8:1:1 ratio. The decoder network used in Task 3 is identical to that in Task 2, except the output dimension is set to 5.

D.3 Pretraining with DC voltage Prediction Task

We pretrained DeepGen_p using the DC voltage prediction task proposed in [40, 47]. In the original work, the authors used two circuit topologies for pretraining: a resistor ladder and a simple operational amplifier (op-amp). They also trained separate GNNs for each topology.

However, to evaluate whether this supervised pretraining approach generalizes effectively, the pretraining process should include more than two circuit topologies simultaneously. In our setting, we pretrained a single GNN on seven different circuit topologies: one resistor ladder, one current mirror, three simple RLC circuits, and two op-amps. We used the same GNN architecture described in [40, 47] to ensure a fair comparison.

D.4 Hyperparameters

Training parameters	DICE Pretraining	SimSiam Pretraining	NT-Xent Pretraining	DeepGen Pretraining	Downstream Task 1	Downstream Task 2	Downstream Task 3
learning rate	0.0003	0.0003	0.0003	0.000005	0.00001	0.0001	0.0001
batch size	1024	1024	1024	1024	50	2048	1024
epochs	200	200	200	200	20000	300	300
(τ_p, τ, τ_n)	(0.2, 0.05, 0.05)	(−, 0.05, −)	—	—	—	—	—
optimizer	Adam	Adam	Adam	Adam	Adam	Adam	Adam

Hyperparameters	DICE	Encoder	Decoder	DeepGEN	ParaGraph
initial node feature dimension	9	—	—	15	7
initial edge feature dimension	5	—	—	1	5
hidden dimension	256	512	256	512	512
dropout probability	0.2	—	0.3	—	—
activation	GELU	GELU	GELU	GELU	GELU
GNN type	GIN	GIN	—	GAT	HGAT
GNN depth	2	depends	0	3	3

D.5 Compute Resources

All experiments were conducted on a server running Ubuntu 20.04.6 LTS with Linux kernel 5.15. The machine is equipped with an Intel(R) Core(TM) i7-9700 CPU operating at 3.00 GHz, 62 GB of RAM, and a single NVIDIA Quadro RTX 6000 GPU with 24 GB of VRAM.

The DICE pretraining stage was executed on a single RTX A6000 GPU and required approximately 2 hours of wall-clock time per trial. Each downstream task was run on a single GPU as well, with Task 1 taking around 1 hour, Task 2 approximately 3 hours, and Task 3 about 8 hours per trial.

E Limitations and Future Work

Pretraining graph models that generalize across both analog and digital circuits remains in its early stages. While we believe this work opens several promising research directions, it also has limitations that suggest avenues for future exploration.

First, more comprehensive comparisons could be made between different graph construction methods. This is a challenging problem in its own right and should be analyzed with the same rigor as seen in recent work on molecules [74]. Evaluations such as graph isomorphism tests may offer deeper insights into optimal circuit-to-graph mapping strategies. Moreover, the definition of “devices” in this work could be broadened to include logic gates, allowing investigation into whether treating logic gates as fundamental units improves modeling efficiency.

Second, exploring alternative machine learning techniques could further enhance performance. For instance, graph transformers may better capture global structural information. In addition, other unsupervised graph representation learning methods—such as graph autoencoders or node/edge masking techniques—hold promise for future work.

Third, addressing more complex EDA tasks would provide stronger validation of the effectiveness of DICE. While this work focuses on graph-level prediction tasks, analog circuit tasks like device parameter optimization [29, 75–77] pose greater challenges. Similarly, digital circuit tasks such as PPA estimation, timing analysis, and congestion prediction involve significantly larger and more complex circuits than those used in this study. Exploring these tasks would open valuable new directions for future research.

F Societal Impacts

Our work contributes to society by enabling more time- and cost-efficient circuit design. By reducing the need for labeled data and simulations, it can accelerate design workflows and make machine learning more accessible within the EDA community. Importantly, we view such automation as a collaborative tool that supports hardware designers, rather than a replacement for human expertise.

However, increased reliance on automation may lead to a decline in fundamental hardware understanding among future engineers and raise concerns about job displacement for traditional designers. Therefore, it is essential to ensure that efforts like ours are developed and deployed with the goal of empowering—rather than replacing—human designers.